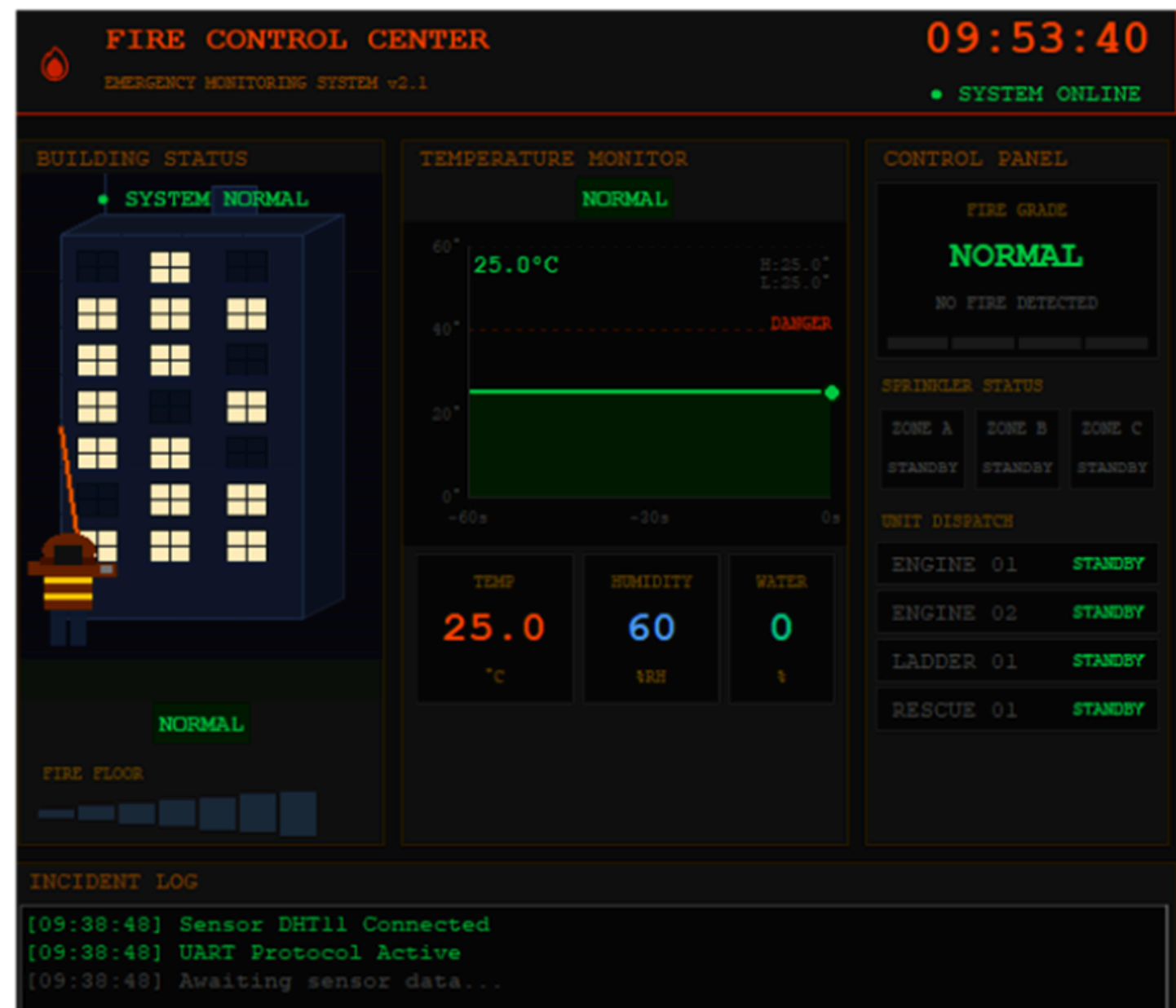


05. 스마트 화재 진압 시스템

프로젝트 개요



STM32 기반 스마트 화재 진압 시스템

- DHT11 센서를 이용한 온습도 측정
- STM32와 PC 간 UART 패킷 통신 구현
- PC GUI를 통한 실시간 화재 상태 모니터링
- 온도 조건에 따른 Fan 제어 및 화재 상태 표시

프로젝트 목표

센서 측정

DHT11 온습도 데이터 수집

패킷 통신

구조화된 데이터 송수신

데이터 검증

CRC8로 수신 데이터 확인

상태 표시

LCD 및 PC GUI 출력

화재 대응

Fan 제어 및 Alert 표시

개발 환경

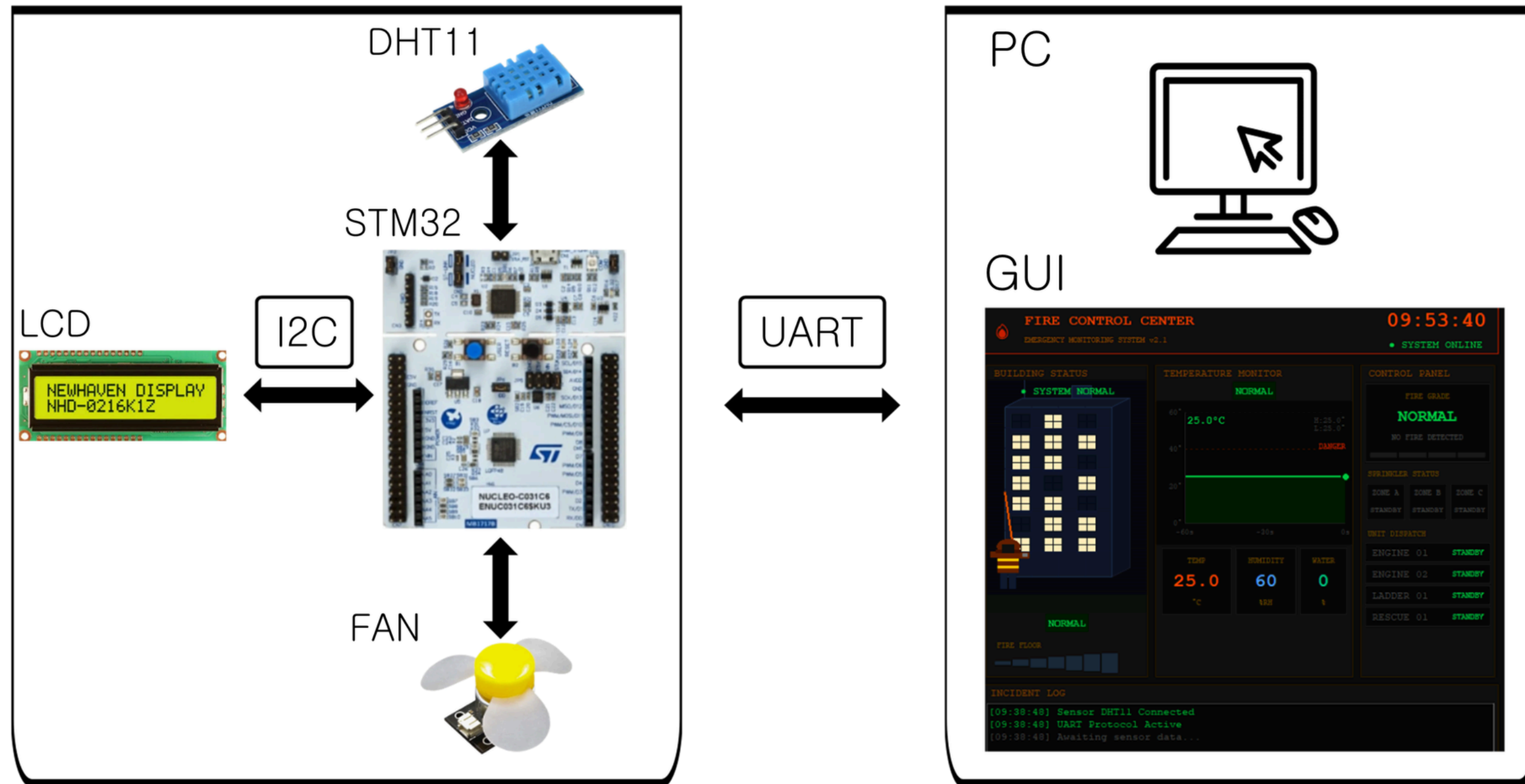


STM32F411re

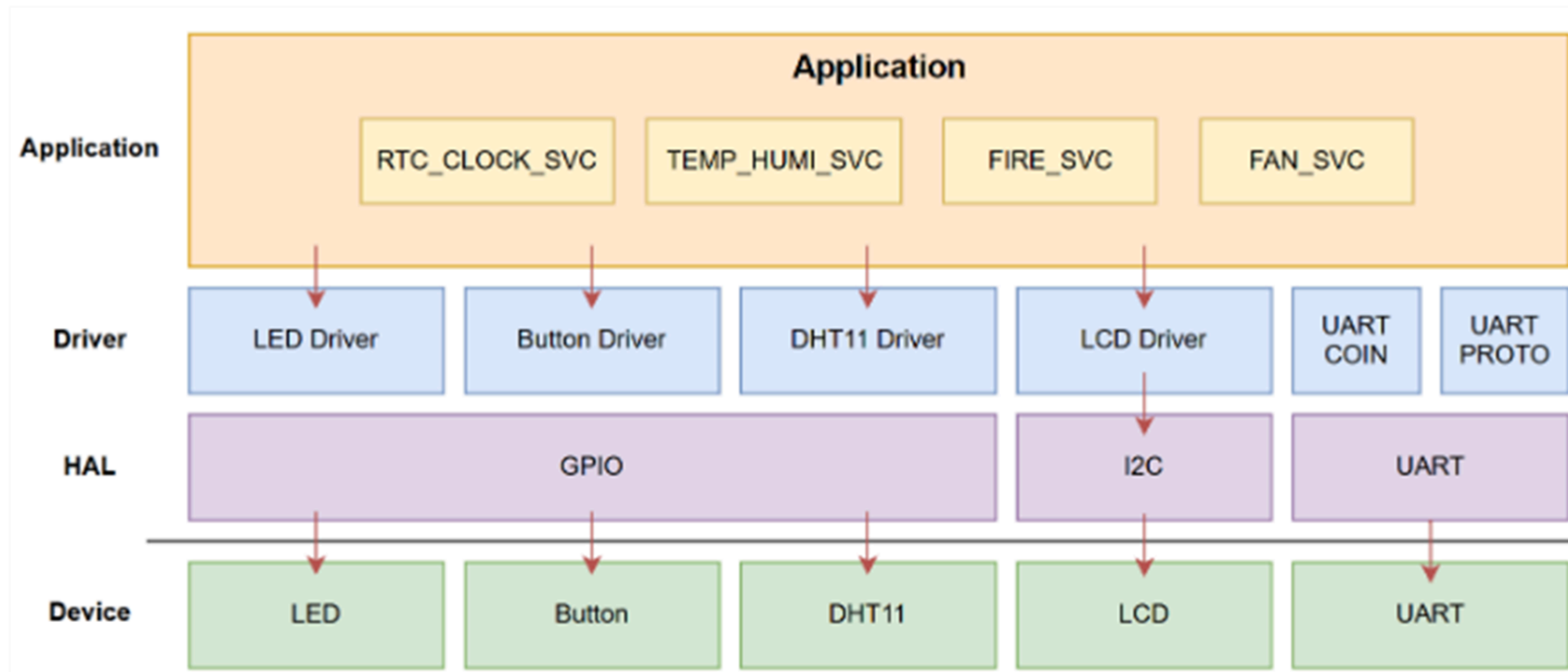
STM32
CubeIDE



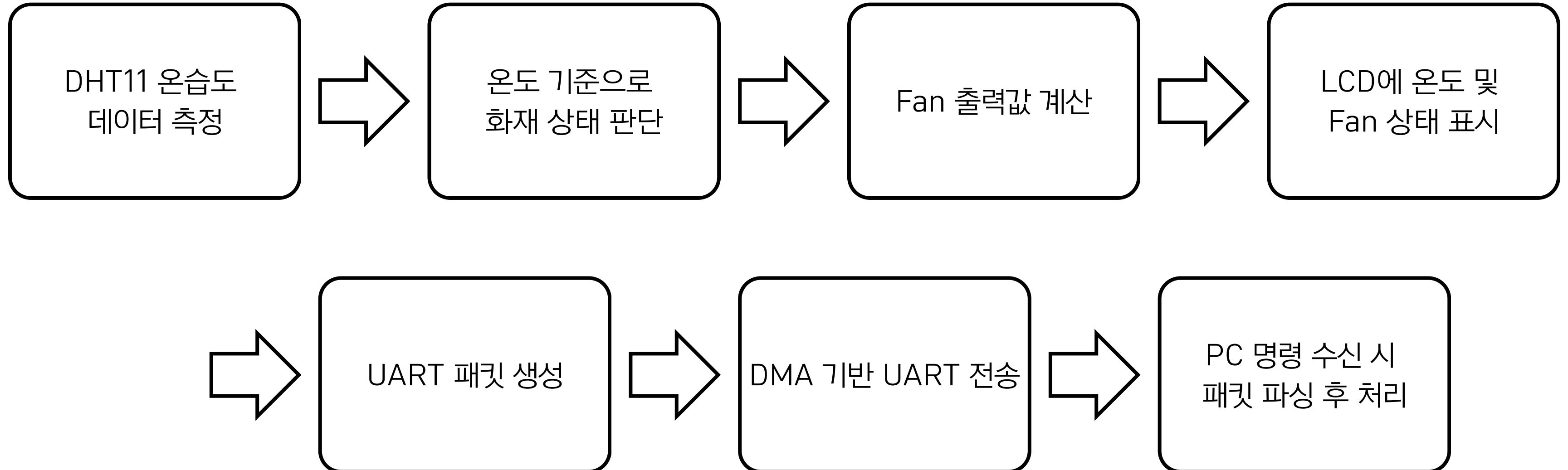
전체 시스템 구조



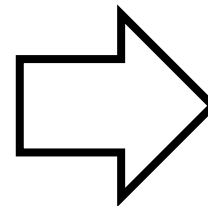
소프트웨어 구조



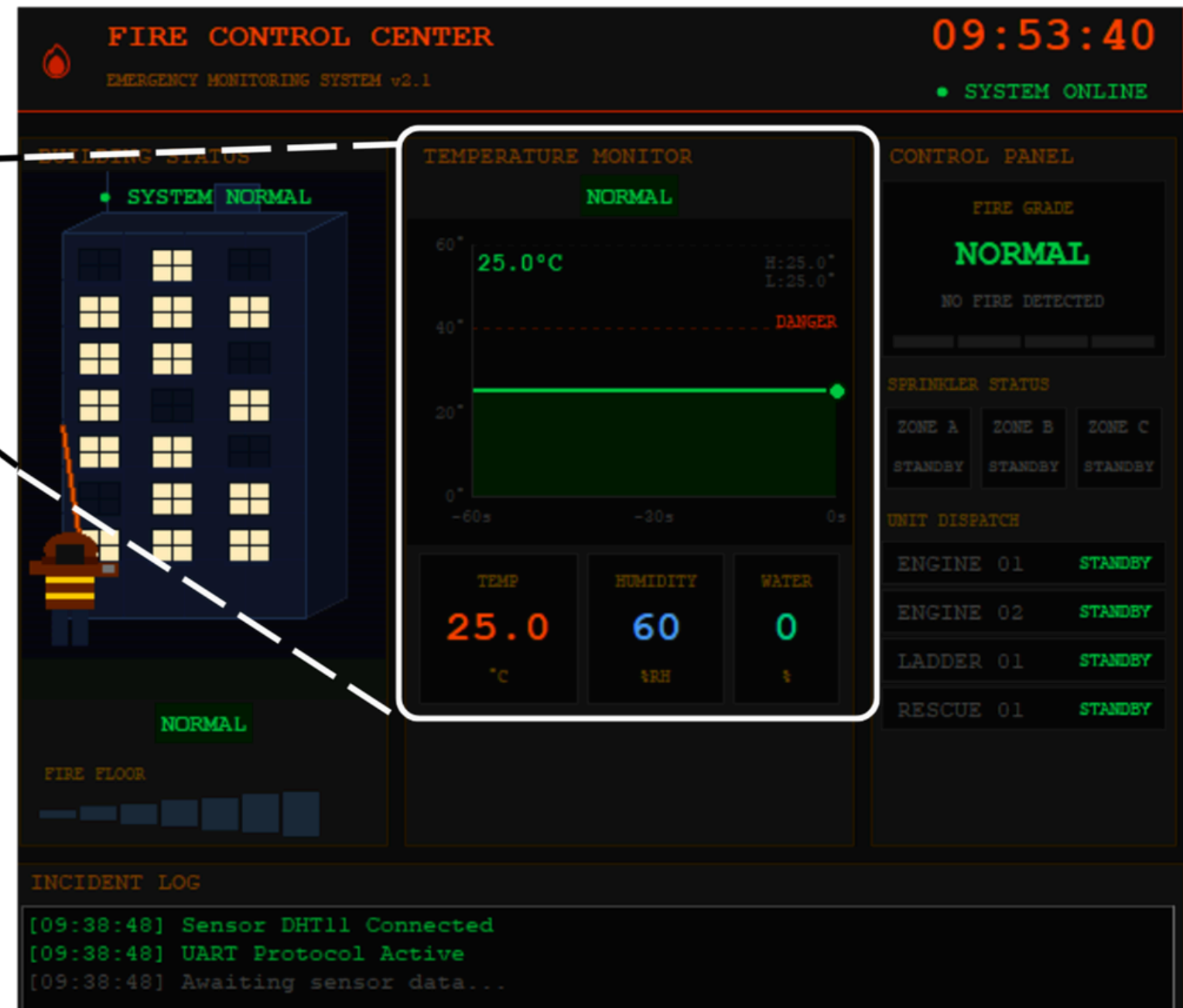
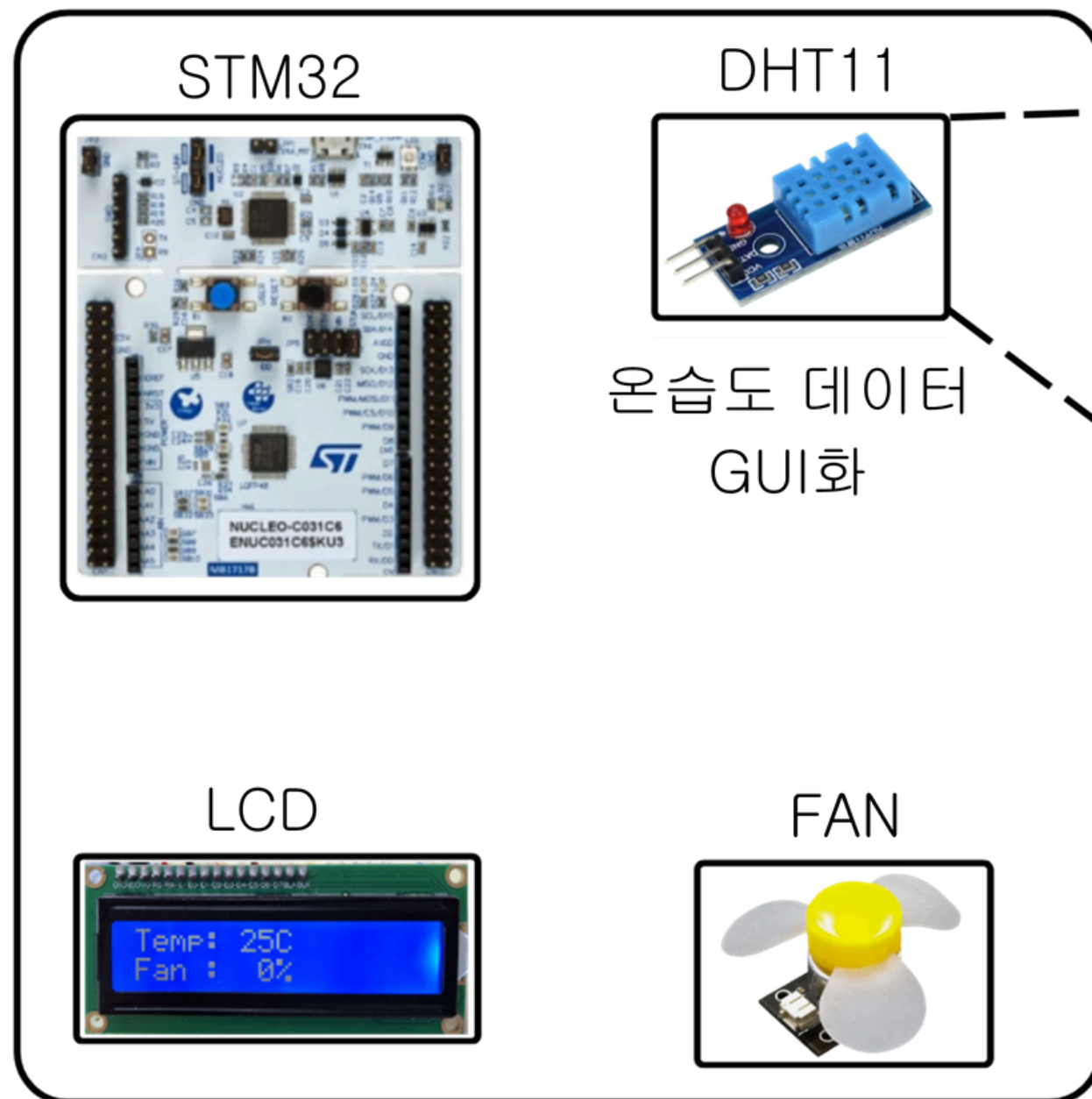
펌웨어 동작 흐름



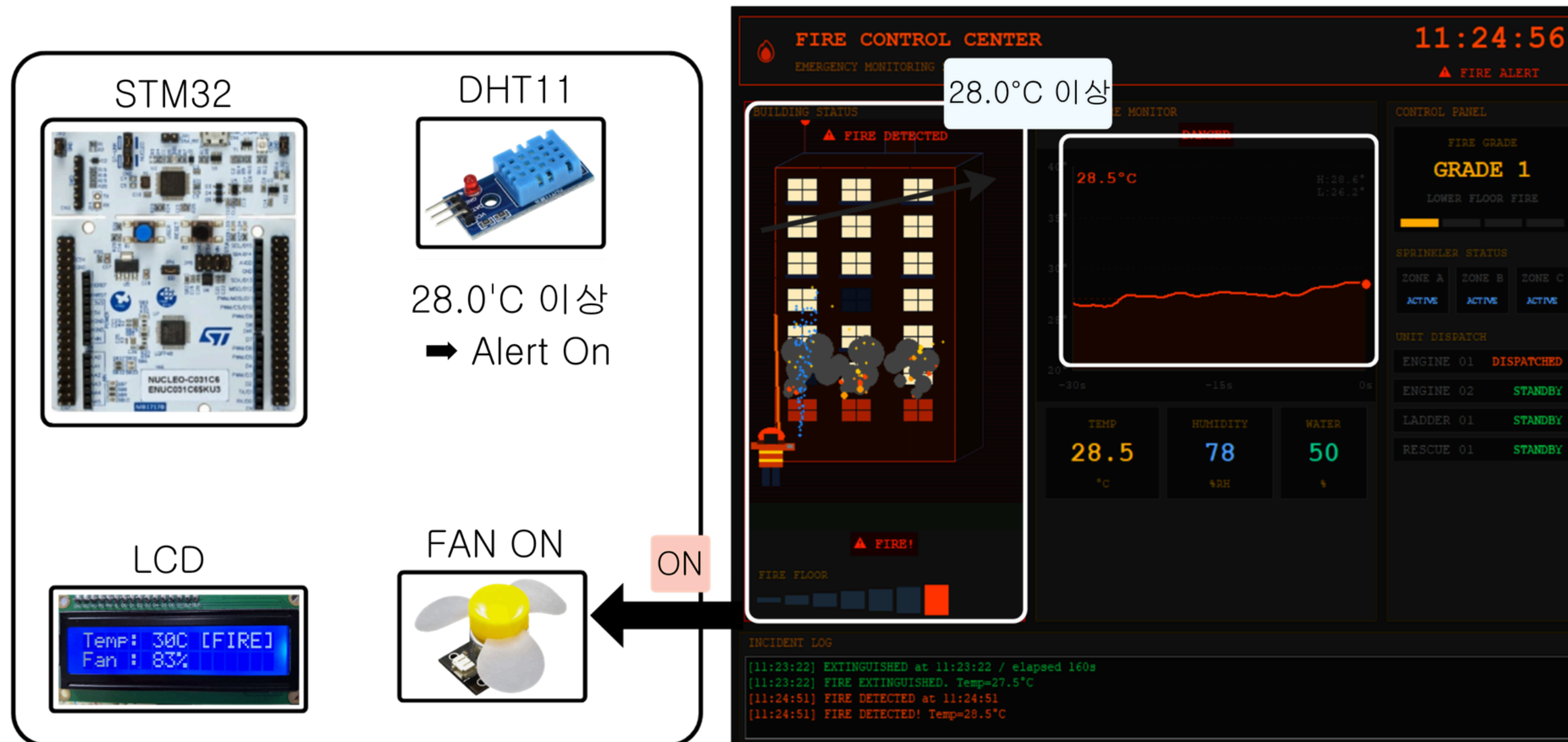
PC GUI 모니터링



동작 설명 1: 온습도 관찰



동작 설명 2: 화재 식별 및 Fan 제어



모니터링 시스템 구성

모니터링 시스템 - 3요소

LCD - [온도 , 팬]



UART - [온도, 습도, 팬, 시간, 상태] 로그

```
[RX] SENSOR_DATA temp=27.9°C humi=71% fan=0% state=0  
[RAW] payload=[27, 8, 72, 0, 0]  
[RX] SENSOR_DATA temp=27.8°C humi=72% fan=0% state=0  
[RAW] payload=[27, 4, 73, 0, 0]
```

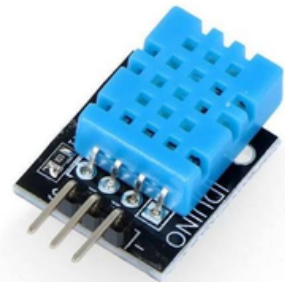
GUI - 화재 발생 및 진압 알림

```
[11:31:42] FIRE DETECTED at 11:31:42  
[11:31:42] FIRE DETECTED! Temp=28.0°C  
[11:33:11] EXTINGUISHED at 11:33:11 / elapsed 89s  
[11:33:11] FIRE EXTINGUISHED. Temp=27.4°C
```

GUI - log 요소 1. 화재 감지 시각 및 온도
2. 화재 진압 시각 및 소요 시간
3. 진압 후 온도



요소기술: DHT11

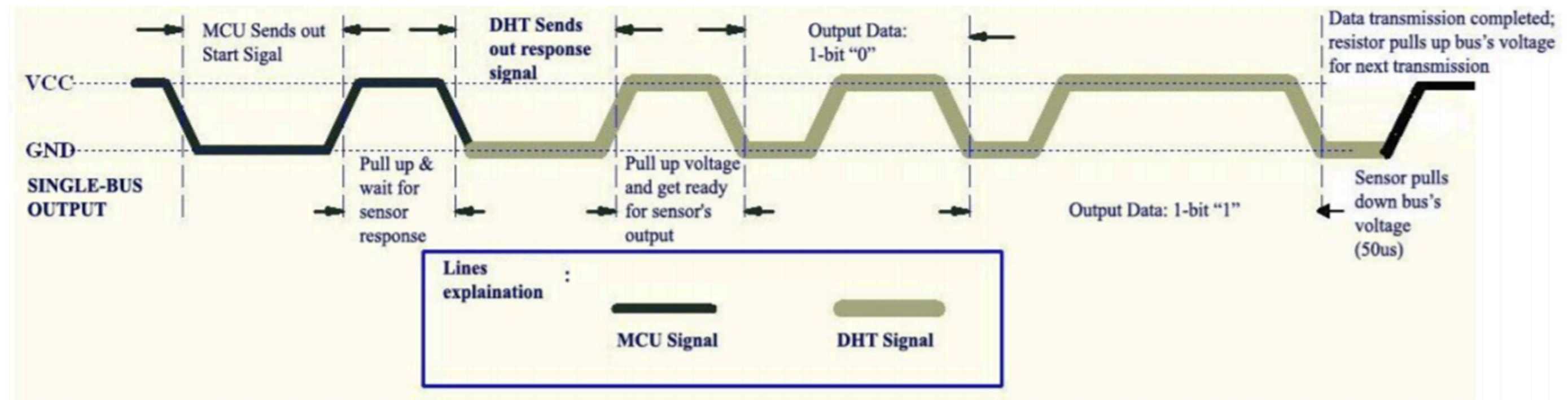


온도, 습도를 측정하는 센서

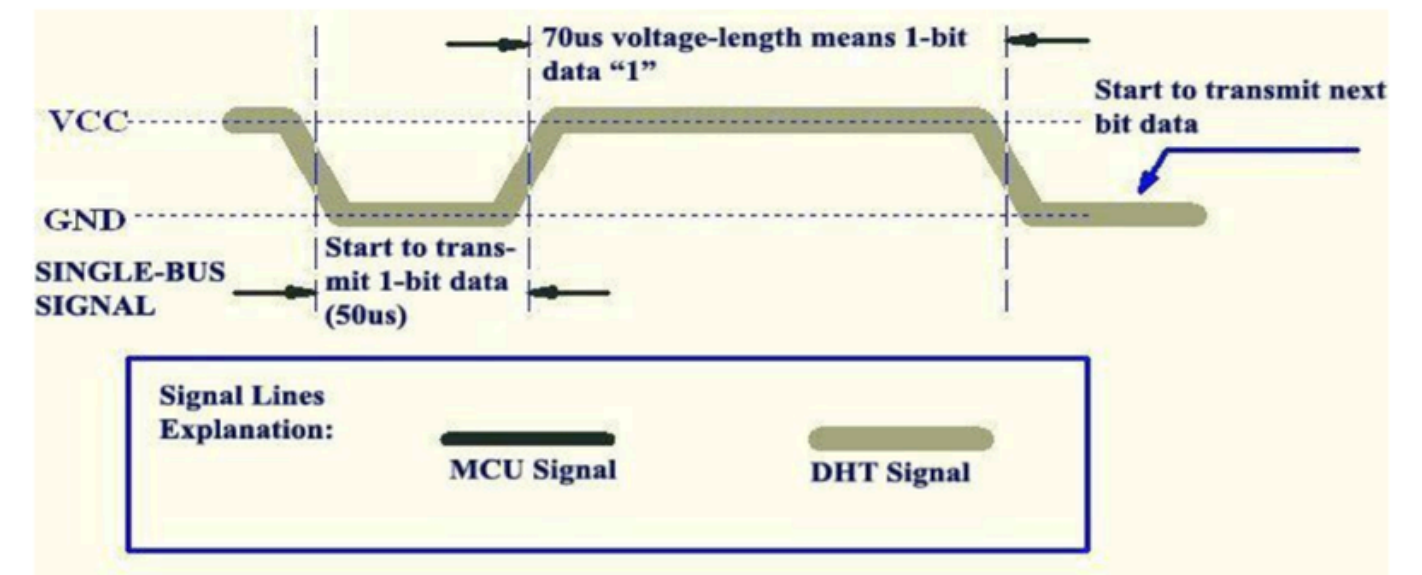
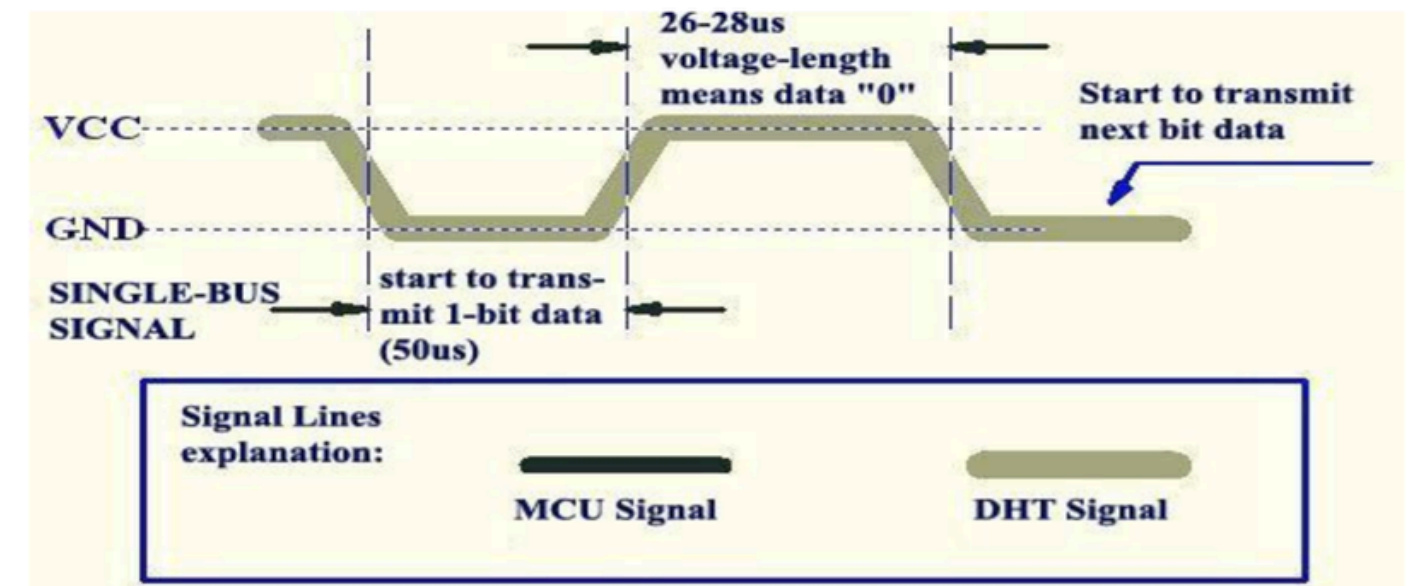
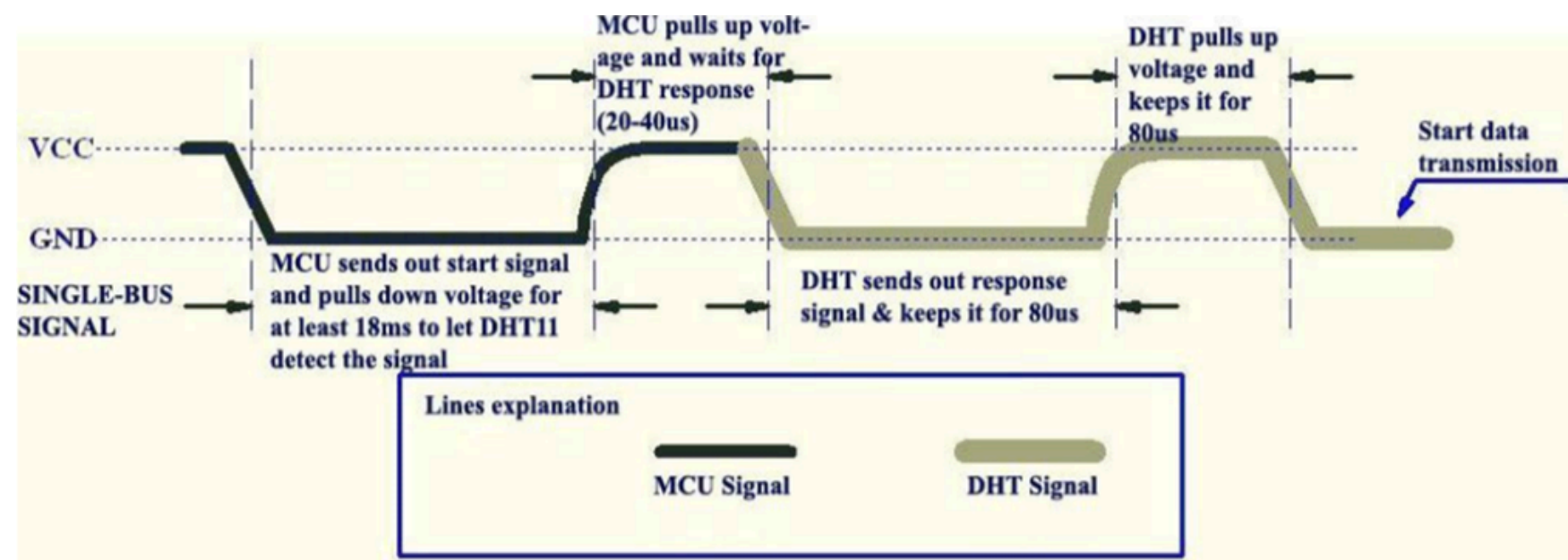
GPIO pin 하나로 MCU와 통신

Output mode에서 MCU가 온습도 정보를 요청하면

Input mode로 변경 후 온습도 정보를 송신 (MCU기준)

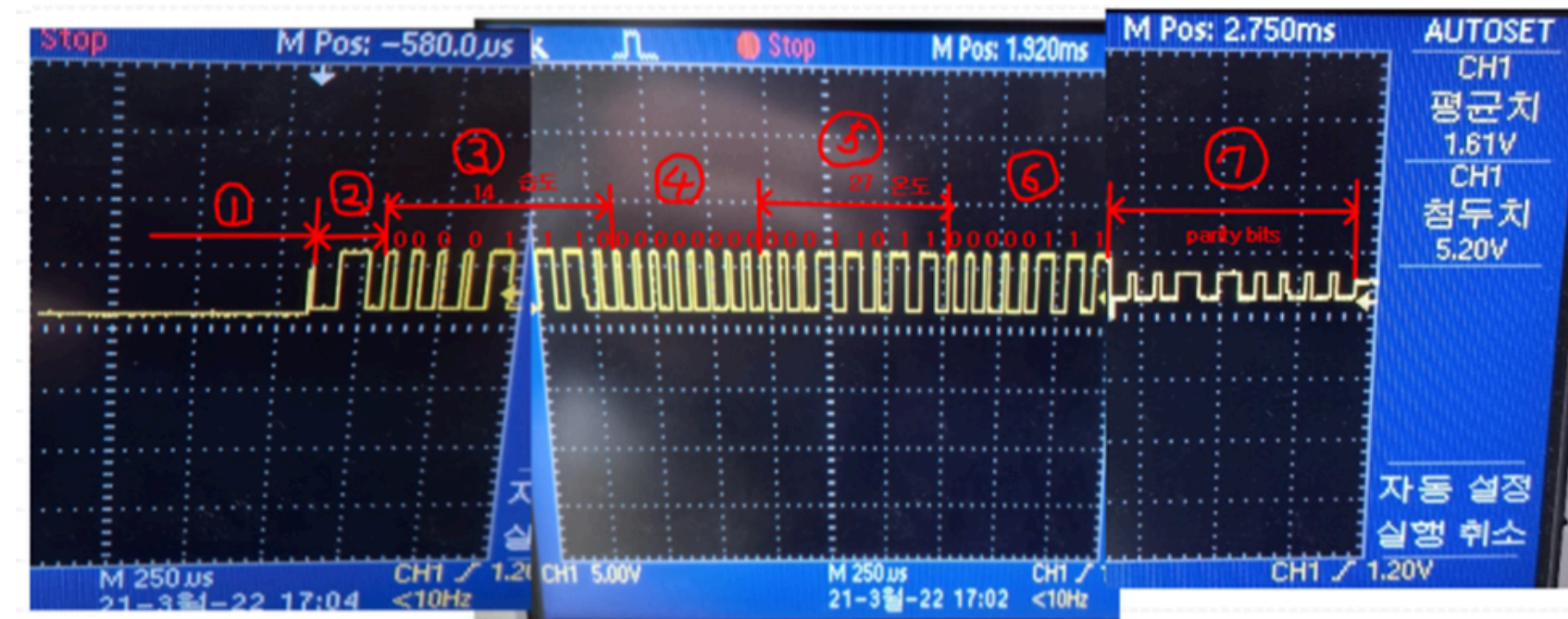
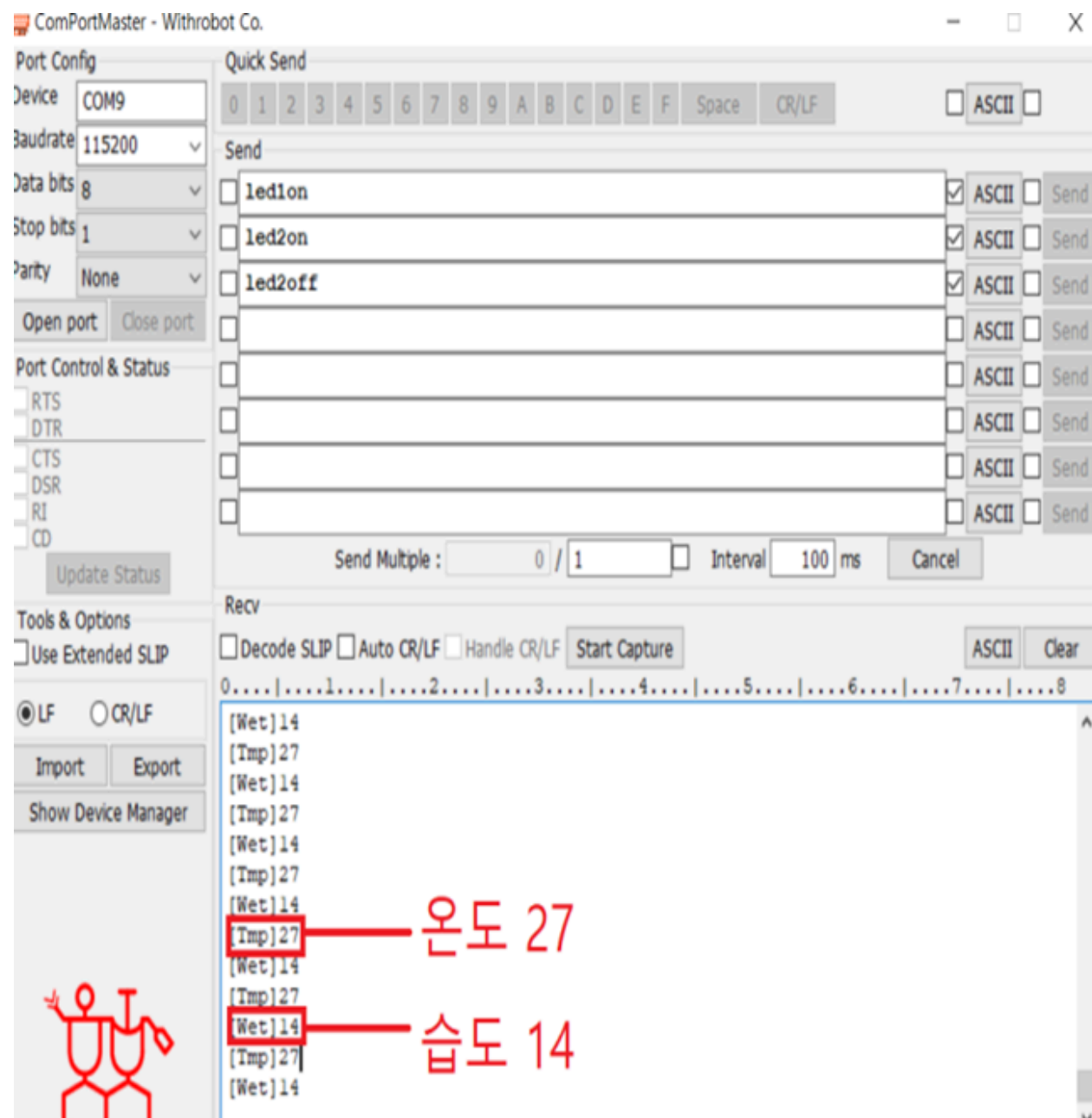


요소기술: DHT11



- MCU에서 Start Signal을 보낸다
(PullDown후 최소 18ms간 LOW신호를 DHT11에 보내준다)
그리고 Pull-up후 High신호를 20~40us동안 보내준다.
- 이후 DHT11은 LOW신호를 80us동안 보낸 다음
80us동안 HIGH신호를 보낸 다음 데이터 전송을 시작 한다.

요소기술: DHT11

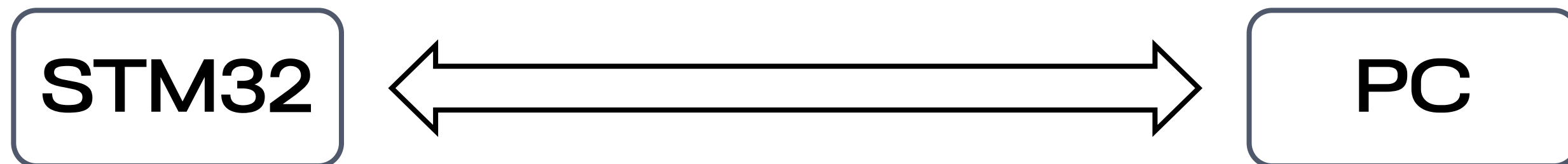


- (1) MCU --> DHT11 start signal
- (2) MCU <-- DHT11 start signal Response
- (3) High humidity 8 bits : 14
- (4) Low humidity 8 bits : 0
- (5) High Temperature 8 bits : 27
- (6) Low Temperature 8 bits : 7
- (7) check sum (상기 data 기준) : $14 + 0 + 27 + 7 = 48$ (00110000)

패킷 통신 구조

항목	SOF (Start of Frame)	LEN (Length)	CMD (Command)	PAYLOAD	CRC8 (Check)
크기	1Byte	1Byte	1Byte	0~32Byte(가변)	1Byte
설명	데이터의 시작 신호 0XAA	payload의 길이	명령 종류 (온도 요청, 팬 제어)	실제 데이터 값	검사용 숫자 (XOR)

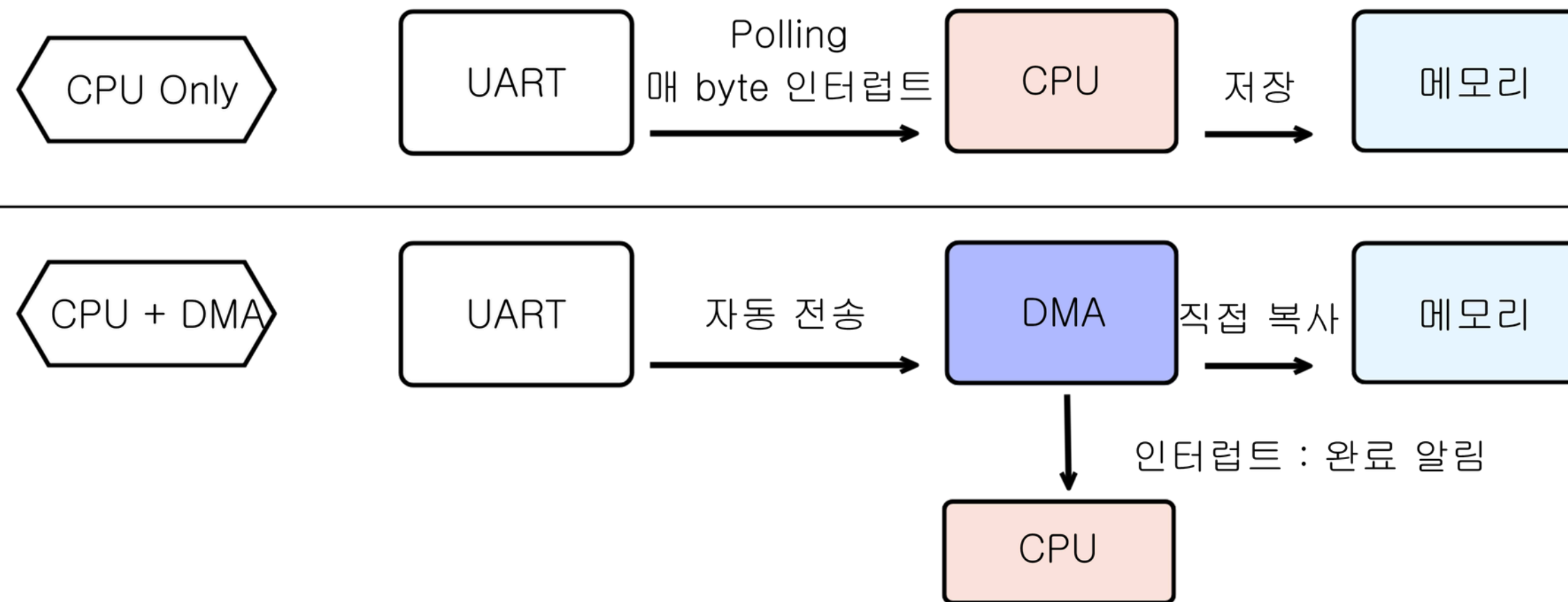
단순 문자열 전송이 아니라, 명령과 데이터를 구분할 수 있는 구조화된 통신 방식 사용



CMD 명령어 구조

카테고리	명령어 범위	주요명령
TEMP_HUMID	0X10~0X1F	CMD_REQUEST_DATA
RTC	0X20~0X2F	CMD_GET_TIME
FAN	0X30~0X3F	CMD_FAN_SET_SPEED
FIRE	0X40~0X4F	CMD_FIRE_DETECTED
RESPONSE	0X81~0X8F	CMD_FIRE_STATUS

DMA 기반 UART 수신



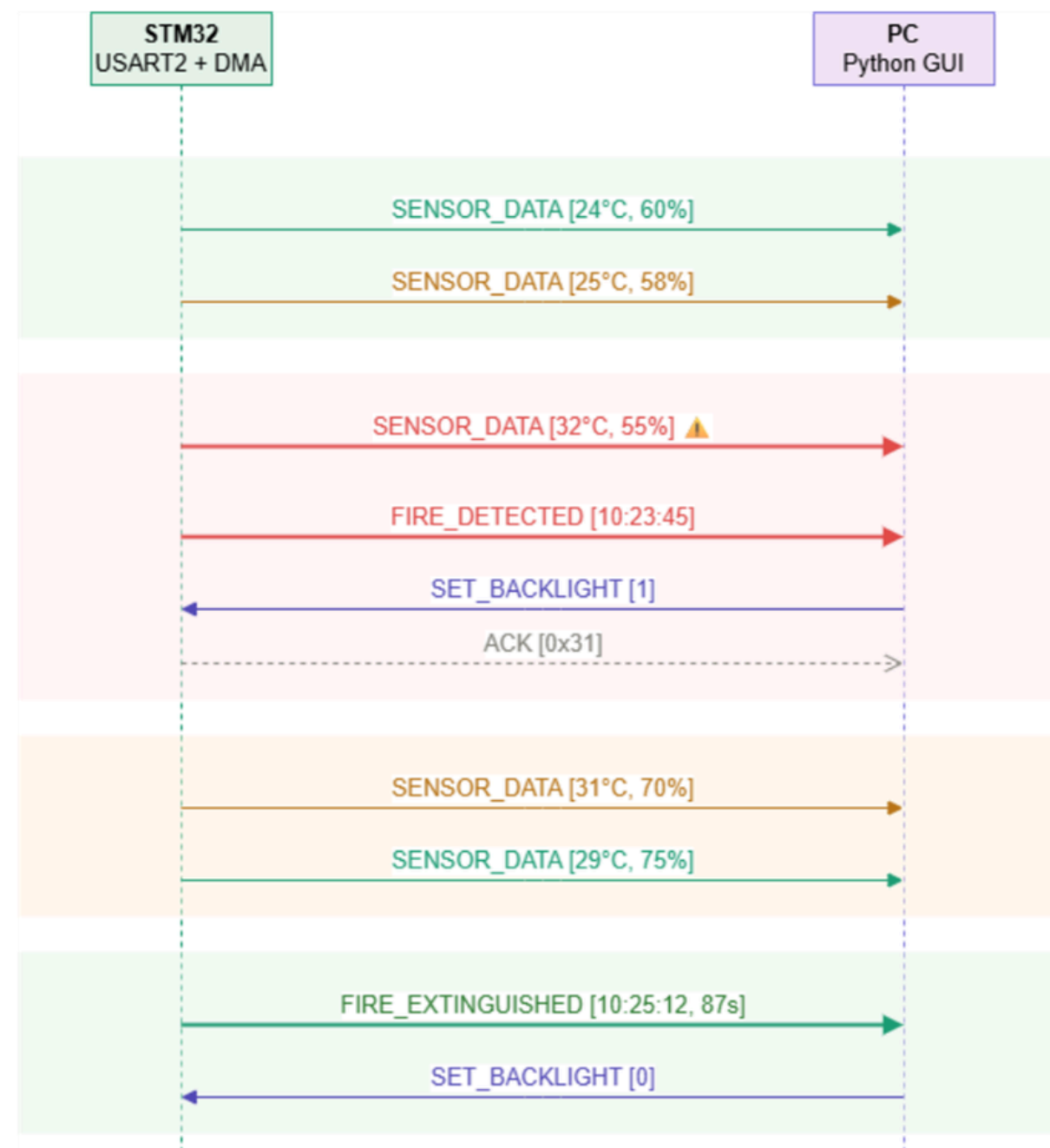
CPU Only : CPU 사용량 증가 및 여유 부족

CPU + DMA : "패킷 파싱 + 검증" 시에만 CPU 활용 ➡ CPU 사용량 감소, 여유 확보

DMA(Direct Memory Access)

- UART 수신 데이터를 메모리로 자동 저장
- CPU는 모든 byte를 직접 처리하지 않음
- 패킷 파싱과 검증 시점에 CPU 사용
- CPU 부하 감소 및 수신 안정성 확보

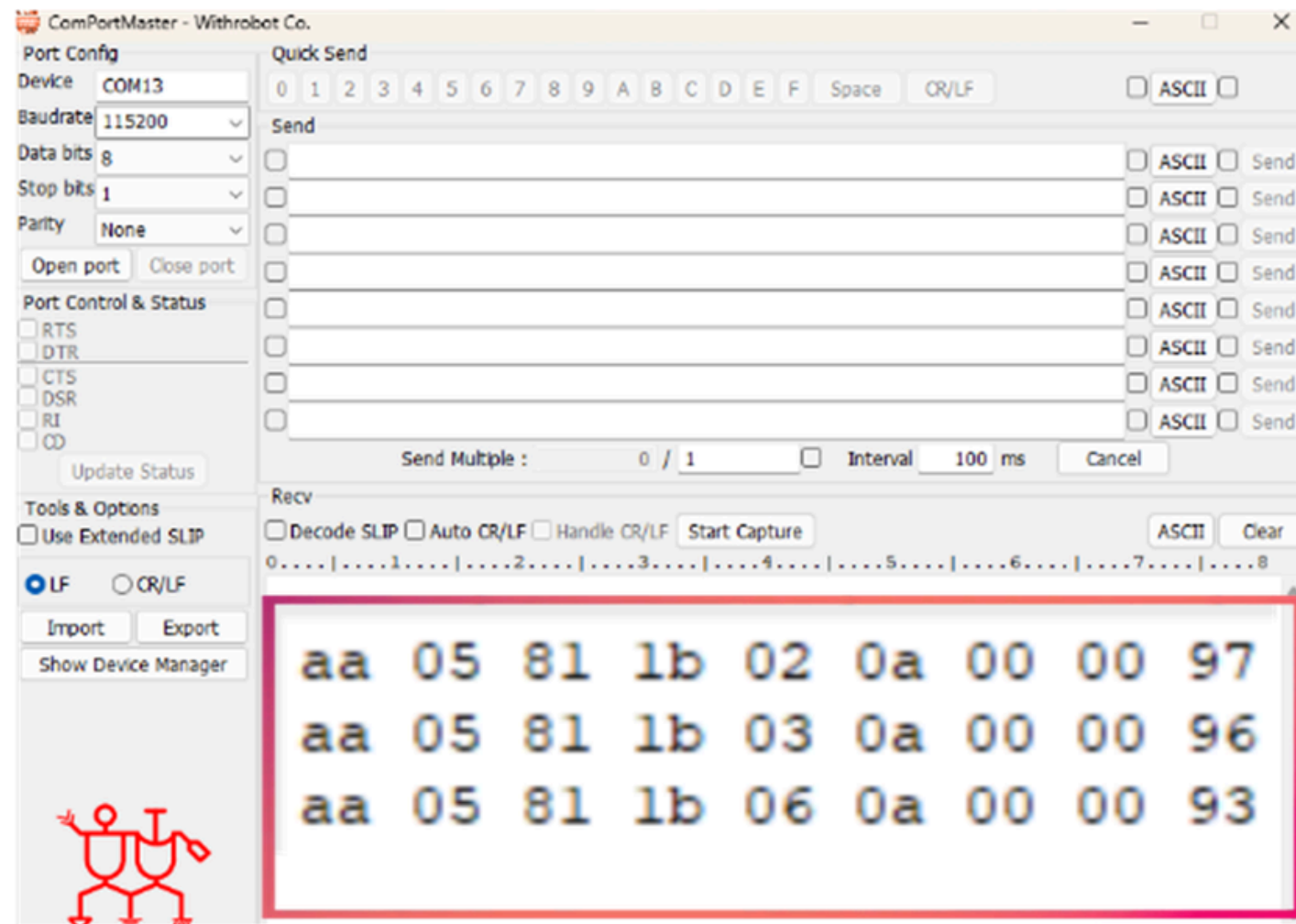
STM32 → PC UART Sequence



상태	트리거	주요 동작
정상 모니터링	주기적으로 호출	DHT11 데이터 측정 및 PC 전송, GUI 그래프 갱신
화재 감지	temp ≥ 28°C	화재 판정, LCD FIRE 신호 ON, GUI 경보 및 시각화
소화 중	화재 상태 지속	스프링클러(Fan) 가동, 온도 비례 출력 제어, 습도 변화 추적
진화 완료	temp < 28°C	진화 판정 FIRE 신호 OFF, 복귀 로그 기록 및 상태 초기화

실제 패킷 송수신 예시

ComPortMaster 패킷 출력 화면



패킷 전송 예시

센서 데이터 전송 (STM32 → PC)



온도 42°C, 습도 55%, FAN 100%

화재 감지 알림 (STM32 → PC)



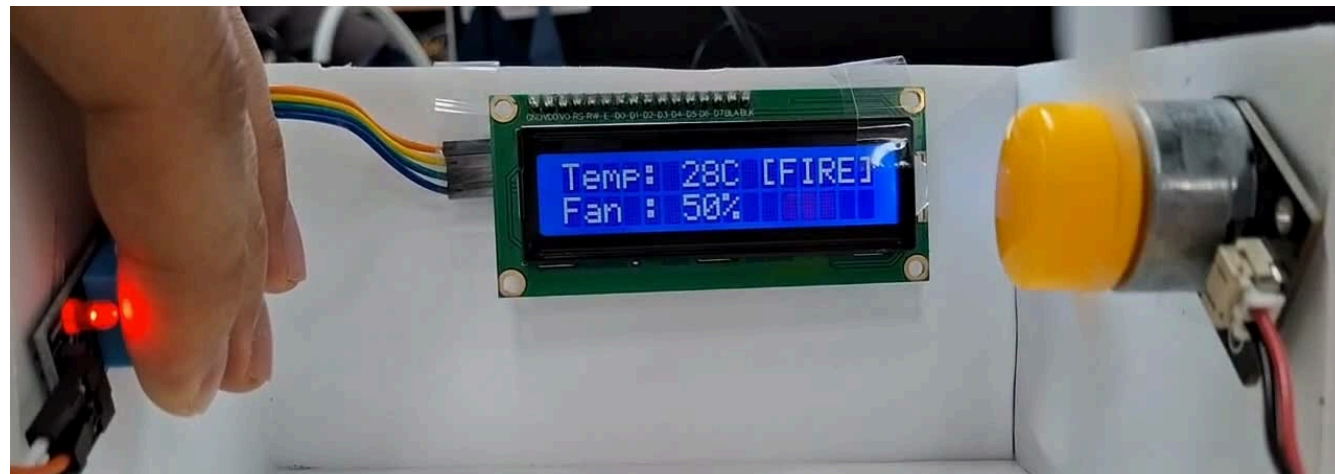
10:23:45 화재발생감지

화재 판단 및 Fan 제어 로직

온도 조건	상태	Fan	GUI
Temp < 28°C	Normal	0%	System Normal
Temp ≥ 28°C	Fire Alert	50% 이상	Fire Detected
Temp 상승	Suppression	온도 비례 증가	Fire Grade 상승
Temp < 28°C 복귀	Extinguished	0%	Fire Extinguished

- 온도 기준값을 초과하면 화재 상태로 전환
- 온도 상승 정도에 따라 Fan 세기 증가
- 화재 상태가 지속되면 GUI와 UART 로그 유지
- 온도가 기준값 이하로 내려가면 진화 완료로 판단

동작 영상 및 결과 확인



온도 ≥ 28 도 $\rightarrow$ 팬 작동



온도 상승 $\rightarrow$ 팬 속도 증가



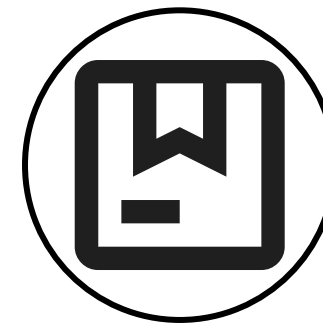
온도, 시간 데이터 전송 및 화재판단 GUI

펌웨어 개발을 통해 얻은 역량



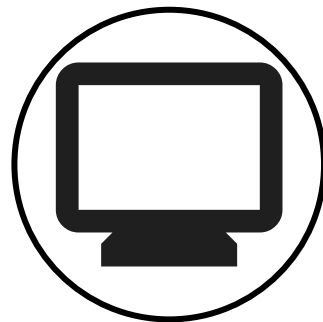
Sensor Data

LCD, Fan, GUI를 연동한 상태 표시 및
출력 제어 경험



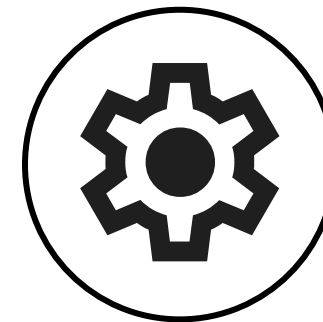
Packet Protocol

센서 데이터를 구조화된 패킷으로
전송하는 방식 이해
SOF, LEN, CMD, CRC를 활용한
통신 안정성 확보



UART + DMA

UART + DMA 기반 수신 처리
구조 경험



Debugging

문제 발생 시 패킷 시작점, 길이, CRC를
기준으로 원인 분석